Einführung in die Programmiersprache Rust

Embedded Systems Engineering – Vorlesung

Prof. Dr.-Ing. Sebastian Schlesinger

Fachbereich 2 – Duales Studium Wirtschaft & Technik
HWR Berlin

Wintersemester 2026

Gliederung

1 Motivation und Geschichte

2 Typsystem und Grundkonstrukte

3 Ownership, Borrowing und Lifetimes

4 Enums, Pattern Matching und Fehlerbehandlung

5 Structs, Traits und Generics

6 Funktionale Elemente

7 Rust für Embedded und High-Performance

8 Zusammenfassung

Motivation und Geschichte

Warum Rust? – Die Probleme von C

Bekannte C-Schwächen:

- Speicherfehler: Buffer Overflows, Use-after-Free, Dangling Pointer, Double Free
- Undefined Behavior: stille Korrumpierung statt kontrolliertem Fehler
- Kein Modulsystem: nur `#include` und Linker
- Kein echtes Typsystem für Generizität – nur `void*` mit Casts
- Manuelle Ressourcenverwaltung: vergessenes `free()`, Memory Leaks

Was Rust verspricht:

- Memory Safety ohne GC – zur Compile-Zeit garantiert
- Zero-Cost Abstractions: Generics, Iteratoren, Pattern Matching – ohne Overhead
- Data Race Freedom: Compiler verhindert gleichzeitige mutable Zugriffe
- Modernes Tooling: Cargo (Build + Deps), Clippy (Linter), Rustfmt
- C-äquivalente Performance: kein Runtime, kein GC, LLVM-Backend

Leitsatz

Rust bietet die Kontrolle von C – mit den Sicherheitsgarantien moderner Sprachen.

Geschichte und Ökosystem

Entwicklungsgeschichte:

- 2006 – Graydon Hoare startet Rust als persönliches Projekt bei Mozilla
- 2009 – Mozilla sponsert offiziell die Entwicklung
- 2015 – Rust 1.0 – erste stabile Version, Stabilität durch *Editions*-System
- 2021 – Gründung der Rust Foundation (AWS, Google, Huawei, Microsoft, Mozilla)
- 2022 – Linux-Kernel akzeptiert Rust als *zweite Sprache* neben C
- 2024+ – Rust in Android, Windows-Kernel, Chromium, kritischer Infrastruktur

Editions (Kompatibilität)

2015 – Erstausgabe
2018 – async/await, Module
2021 – Closures, Patterns
2024 – neueste Edition

Verschiedene Editions sind *innerhalb* eines Projekts mischbar!

Toolchain

`rustup` – Installer/Updater
`cargo` – Build + Paketmanager
`rustc` – Compiler (LLVM)
`clippy` – Linter
`rustfmt` – Autoformatter

Vergleich: Hello World in C und Rust

C:

```c
#include <stdio.h>

int main(void) {
    printf("Hello, ESE!\n");
    return 0;
}
```

Kompilierung:

```
gcc -std=c11 -Wall -o hello
hello.c
./hello
```

Rust:

```rust
fn main() {
    println!("Hello, ESE!");
}
```

Kompilierung:

```
cargo new hello && cd hello
cargo run
```

Was fällt auf?

- Kein `#include` – Modulsystem mit `use`
- `println!` ist ein *Makro* (`!`)
- Kein explizites `return 0` nötig
- Cargo erzeugt Projektstruktur automatisch

Typsystem und Grundkonstrukte

Variablen, Mutabilität und Typen

Variablen und Bindungen:

```rust
// Immutable per Default!
let x = 42;
// x = 43;  // Fehler!

// Explizit mutable
let mut counter = 0;
counter += 1;  // OK

// Shadowing (Neubindung)
let x = x + 1;          // OK: neues x
let x = "jetzt Text";  // anderer Typ!

// Konstanten (zur Compilezeit)
const MAX_SIZE: usize = 128;
```

Skalare Typen:

i8..i128	vorzeichenbeh.	fest
u8..u128	vorzeichenlos	fest
isize/usize	ptr-breit	arch.
f32, f64	Gleitkomma	
bool	true/false	
char	4 Byte Unicode	

Zusammengesetzte Typen:

```rust
// Tupel (heterogen, feste Länge)
let t: (i32, f64, bool) = (42, 3.14, true);
let x = t.0;  // 42

// Array (homogen, feste Länge)
let arr: [u8; 4] = [0xFF; 4];

// Slice (dynamisches Fenster)
let slice: &[u8] = &arr[1..3];
```

Kontrollstrukturen und Ausdrücke

if ist ein Ausdruck:

```
let voltage = 3.3;
let status = if voltage > 3.0 {
    "OK"
} else if voltage > 2.0 {
    "Warnung"
} else {
    "Kritisch"
};  // Ergebnis direkt zuweisbar
```

Schleifen:

```
for i in 0..10 { println!("{}", i); }

while !uart_ready() { /* warten */ }

// loop mit break-Wert
let result = loop {
    let v = adc_read();
    if v > THRESHOLD { break v; }
};
```

Blöcke als Ausdrücke:

```
// Letzter Ausdruck ohne ; = Rückgabewert
let y = {
    let a = 5;
    let b = 10;
    a + b   // = 15, Wert von y
};
```

Paradigmenwechsel gegenüber C

- C: Anweisungen dominieren; Rust: *fast alles* ist ein Ausdruck
- `if`, Blöcke, `match`, `loop` – alle liefern Werte
- Kein ternärer Operator nötig – `if` ersetzt `?:`
- Kein impliziter Fallthrough wie bei C-`switch`

Funktionen und Typinferenz

```rust
// Funktionssignatur: Typen explizit
fn clamp(val: i32, lo: i32, hi: i32) -> i32 {
    if val < lo { lo }
    else if val > hi { hi }
    else { val }  // letzter Ausdruck = return
}

// Unit-Rückgabetyp (wie void in C)
fn greet(name: &str) {
    println!("Hallo, {}!", name);
}

// Frühes return ist möglich
fn divide(a: f64, b: f64) -> Option<f64> {
    if b == 0.0 { return None; }
    Some(a / b)
}

// Generische Funktion
fn max_of<T: PartialOrd>(a: T, b: T) -> T {
    if a >= b { a } else { b }
}
```

Typinferenz:

```rust
let x = 42;          // i32 (Default)
let y = 3.14;        // f64 (Default)
let z = x as f64 + y;  // f64

// Turbofish bei Mehrdeutigkeit
let v = "42".parse::<i32>().unwrap();

// Typaliase
type Volt = f32;
type Register = u32;
let reg: Register = 0x4001_0800;
```

Vergleich zu C

- C: alle Typen explizit, keine Inferenz
- Rust: lokale Inferenz, aber Funktionssignaturen *immer* explizit
- Sicherer: kein impliziter Integer-Overflow (panic im Debug)

Ownership, Borrowing und Lifetimes

Das Ownership-Prinzip

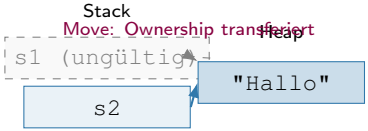
Die drei Regeln:

1. Jeder Wert hat genau einen Owner
2. Wird der Owner ungültig (*out of scope*), wird der Wert automatisch freigegeben
3. Ownership kann transferiert werden (*Move*)

```rust
fn main() {
    let s1 = String::from("Hallo");
    let s2 = s1;  // MOVE: s1 ungültig!
    // println!("{}", s1); // Fehler!
    println!("{}", s2);     // OK
}  // s2 wird hier gedroppt -> Speicher frei
```

Kein free() nötig!

Der Compiler fügt `drop()` automatisch am Scope-Ende ein – keine Leaks, kein Double Free, kein Use-after-Free.



Copy-Typen (Stack-only):

```rust
let a: i32 = 42;
let b = a;    // COPY, nicht Move!
println!("{}", a); // OK: i32 ist Copy
```

Copy vs. Move

Primitive Typen (`i32`, `f64`, `bool`) implementieren `Copy` – werden bitweise kopiert. Heap-Typen (`String`, `Vec`) werden *gemoved*.

Borrowing – Referenzen statt Kopien

Immutable Referenz (`&T`):

```rust
fn length(s: &String) -> usize {
    s.len()  // liest nur, kein Ownership
}
let s = String::from("Rust");
let n = length(&s);  // Borrows s
println!("{} hat {} Zeichen", s, n);
// Mehrere immutable Refs gleichzeitig: OK
let r1 = &s;
let r2 = &s;
```

Mutable Referenz (`&mut T`):

```rust
fn push_excl(s: &mut String) {
    s.push('!');
}
let mut s = String::from("Hallo");
push_excl(&mut s);  // exklusiver Zugriff
println!("{}", s);   // "Hallo!"
```

Borrowing-Regeln (Compile-Zeit!):

1. Entweder *beliebig viele* `&T` oder genau eine `&mut T`
2. Niemals beides gleichzeitig
3. Referenzen dürfen den Owner nicht überleben

```rust
let mut v = vec![1, 2, 3];
let r = &v[0];      // immutable Borrow
// v.push(4);       // Fehler! mut Borrow
//                  // während immut. aktiv
println!("{}", r);
v.push(4);          // jetzt OK
```

Vergleich zu C

C erlaubt beliebig viele Aliasing-Pointer gleichzeitig – Quelle von Data Races und Iterator-Invalidierung. Rust verhindert dies *zur Compilezeit*.

Lifetimes – Gültigkeitsdauern

Problem: Dangling Reference

```
// fn dangling() -> &String {
//     let s = String::from("weg");
//     &s  // Fehler! s wird gedroppt
// }
// In C: UB -- In Rust: Compilerfehler!
```

Explizite Lifetime-Annotation:

```
// 'a = generischer Lifetime-Parameter
fn longest<'a>(x: &'a str,
               y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}
let s1 = String::from("lang");
{
    let s2 = String::from("kurz");
    let r = longest(&s1, &s2);
    println!("{}", r); // OK
}  // s2 weg -> r darf hier nicht mehr leben
```

Lifetime Elision (automatisch):

```
// Compiler ergänzt Lifetimes automatisch:
fn first(s: &str) -> &str { &s[..1] }
// wird intern zu:
// fn first<'a>(s: &'a str) -> &'a str
```

Elision-Regeln (vereinfacht)

1. Jeder Referenz-Parameter bekommt einen eigenen Lifetime
2. Bei genau einem Input-Lifetime → Output erbt ihn
3. Bei `&self` → Output erbt Lifetime von `self`

Kernidee

Lifetimes sind *keine* Laufzeitkosten – sie existieren nur im Typsystem und werden vom Compiler zur Compilezeit geprüft.

Enums, Pattern Matching und Fehlerbehandlung

Enums – mehr als C-Konstanten

Einfaches Enum (wie C):

```rust
enum State { Idle, Running, Error }
let s = State::Running;
```

Enum mit Daten (Tagged Union):

```rust
enum SensorEvent {
    Temperature(f32),
    Humidity(f32),
    Error { code: u8, msg: String },
    Shutdown,
}
let evt = SensorEvent::Temperature(23.5);
```

Vergleich zu C

In C benötigt man `enum` + `union` + manuelles Tag-Feld – fehleranfällig. Rust-Enums vereinen dies typsicher in *einem* Konstrukt (*Algebraic Data Types*).

Die wichtigsten Standard-Enums:

```rust
// Option: Wert oder nichts
enum Option<T> { Some(T), None }

// Result: Erfolg oder Fehler
enum Result<T, E> { Ok(T), Err(E) }

// Kein NULL-Pointer mehr!
// Kein Fehlercode als Rückgabewert!
```

Pattern Matching mit `match`

```rust
fn handle_event(evt: SensorEvent) {
    match evt {
        SensorEvent::Temperature(t) if t > 80.0
            => alarm(t),
        SensorEvent::Temperature(t)
            => log_temp(t),
        SensorEvent::Humidity(h)
            => log_humidity(h),
        SensorEvent::Error { code, msg }
            => eprintln!("E{}: {}", code, msg),
        SensorEvent::Shutdown => shutdown(),
    }
}
```

Exhaustiveness

Der Compiler erzwingt, dass *alle* Varianten behandelt werden. Vergisst man einen Arm: Compile-Fehler – nicht wie C-`switch`.

Pattern-Arten:

```rust
let x = 42;
match x {
    0 => println!("Null"),
    1..=9 => println!("einstellig"),
    10 | 20 | 30 => println!("rund"),
    n @ 40..=50 => println!("nah: {n}"),
    _ => println!("sonstig"),
}
```

```rust
// Destructuring
let (a, b, c) = (1, 2.0, true);

// if-let für einzelne Varianten
if let Some(val) = maybe_value {
    println!("Wert: {}", val);
}
// while-let
while let Some(item) = iter.next() {
    process(item);
}
```

Fehlerbehandlung: `Result` und der `?`-Operator

```rust
use std::fs;
use std::io;

// Ohne ?-Operator (explizit)
fn read_cfg_v1() -> Result<String, io::Error> {
    match fs::read_to_string("cfg.toml") {
        Ok(text) => Ok(text),
        Err(e) => Err(e),
    }
}
// Mit ?-Operator (idiomatisch)
fn read_cfg() -> Result<String, io::Error> {
    let text = fs::read_to_string("cfg.toml")?;
    Ok(text) // ? propagiert Err automatisch
}
```

Fehler-Ketten:

```rust
fn init_sensor() -> Result<Sensor, Error> {
    let cfg = read_config()?;
    let port = open_port(&cfg)?;
    let sensor = calibrate(port)?;
    Ok(sensor)
}
// ? propagiert Fehler automatisch!
```

Vergleich zu C

- C: Fehlercode als `int`, manuelles Prüfen, `goto cleanup`
- Rust: `Result<T,E>` erzwingt Behandlung – vergessen unmöglich
- `?` ersetzt `if (err) return err;`

`Result` ist ein normaler Enum-Wert – kein Stack-Unwinding wie bei C++-Exceptions.

Structs, Traits und Generics

Structs und `impl`-Blöcke

```rust
struct Sensor {
    id: u8, offset: f32, last_value: f32,
}
impl Sensor {
    fn new(id: u8) -> Self { // Konstruktor
        Self { id, offset: 0.0, last_value: 0.0 }
    }
    fn calibrate(&mut self, offset: f32) {
        self.offset = offset;  // &mut self
    }
    fn last(&self) -> f32 { self.last_value }
    fn into_raw(self) -> u8 { self.id } // Move
}
```

Verwendung:

```rust
let mut s = Sensor::new(1);
s.calibrate(0.5);
println!("Sensor {}", s.last());
let raw = s.into_raw(); // s jetzt ungültig
```

Vergleich zum C `self`-Muster

- C: `void Sensor_init(Sensor_t *self)`
- Rust: `fn new(id: u8) -> Self`
- `self`/`&self`/`&mut self` regeln Ownership
- Kein Header – `pub` steuert Sichtbarkeit

Derive-Makros: `#[derive(Debug, Clone, PartialEq)]` – Compiler implementiert Traits automatisch.

Traits – Interfaces und Polymorphismus

Trait-Definition (wie Interface):

```rust
trait Measurable {
    fn read(&mut self) -> f32;
    fn unit(&self) -> &str;
    // Default-Implementierung möglich
    fn report(&mut self) -> String {
        format!("{:.1} {}",
                self.read(), self.unit())
    }
}
```

Implementierung für Typen:

```rust
impl Measurable for Sensor {
    fn read(&mut self) -> f32 {
        self.last_value = hw_read(self.id);
        self.last_value
    }
    fn unit(&self) -> &str { "deg C" }
}
```

Statischer Dispatch (Generics):

```rust
// Monomorphisierung -> Zero-Cost
fn log_reading<T: Measurable>(m: &mut T) {
    println!("{}", m.report());
}
```

Dynamischer Dispatch:

```rust
// Trait Object -> vtable wie in C
fn log_any(m: &mut dyn Measurable) {
    println!("{}", m.report());
}
```

Vergleich: vtable in C vs. Rust

- C: manuelle vtable-Structs, Funktionspointer, unsichere Casts
- Rust: `dyn Trait` erzeugt vtable automatisch – typsicher
- Statisch: kein Overhead (Monomorphisierung)

Generics und Trait Bounds

Generische Strukturen:

```rust
struct RingBuffer<T, const N: usize> {
    data: [Option<T>; N],
    head: usize,
    count: usize,
}
impl<T: Copy, const N: usize>
    RingBuffer<T, N>
{
    fn new() -> Self {
        Self { data: [None; N],
               head: 0, count: 0 }
    }
    fn push(&mut self, val: T) {
        let idx = (self.head+self.count) % N;
        self.data[idx] = Some(val);
        if self.count < N { self.count += 1;
        else { self.head = (self.head+1) % N;
    }
}
```

Trait Bounds – **`where`**-Syntax:

```rust
fn process<T>(items: &[T])
where T: Measurable + Clone + Debug,
{
    for item in items {
        println!("{:?}", item);
    }
}
```

Zero-Cost Abstraction

Generics werden *monomorphisiert*: für jeden konkreten Typ eine spezialisierte Version. ⇒ Kein Laufzeit-Overhead!

Const Generics (seit 1.51):

```rust
let mut buf: RingBuffer<u16, 64>
    = RingBuffer::new();
buf.push(0xCAFE);
```

Const Generics ersetzen C-Makros für größenparametrisierte Buffer – typsicher und ohne Präprozessor.

Funktionale Elemente

Closures – anonyme Funktionen

```rust
// Closure-Syntax
let add = |a: i32, b: i32| a + b;
println!("{}", add(3, 4)); // 7

// Closures fangen Umgebung ein
let offset = 1.5_f32;
let calibrate = |raw: f32| raw + offset;

// Mutable Capture
let mut count = 0;
let mut inc = || { count += 1; count };
println!("{}", inc()); // 1
```

Closure-Traits:

`Fn` – borrows immutabel
`FnMut` – borrows mutabel
`FnOnce` – nimmt Ownership

```rust
// Als Funktionsparameter
fn apply<F: Fn(f32) -> f32>(
    data: &mut [f32], f: F
) {
    for val in data.iter_mut() {
        *val = f(*val);
    }
}
let mut readings = [1.0, 2.0, 3.0];
apply(&mut readings, |x| x * 0.01);
```

Vergleich zu C

C: Funktionspointer + `void *ctx`
Rust: Closures fangen Kontext automatisch ein – typsicher, ohne Indirektion.

Iteratoren und funktionale Ketten

```rust
let data = vec![1, 2, 3, 4, 5, 6, 7, 8];

// Funktionale Kette (lazy!)
let result: Vec<i32> = data.iter()
    .filter(|&&x| x % 2 == 0)  // gerade
    .map(|&x| x * x)           // quadrieren
    .collect();
// result = [4, 16, 36, 64]

let sum: i32 = data.iter().sum();

// Zip (Skalarprodukt)
let a = [1.0, 2.0, 3.0];
let b = [4.0, 5.0, 6.0];
let dot: f64 = a.iter().zip(b.iter())
    .map(|(x, y)| x * y).sum(); // 32.0
```

Das **`Iterator`**-Trait:

```rust
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
    // + 70 Default-Methoden (map, filter, ...)
}
```

Zero-Cost Iteratoren

Die Iterator-Kette wird vom Compiler *zu einer einzigen Schleife* fusioniert. Keine Zwischenallokation, keine virtuellen Aufrufe. ⇒ Äquivalent zu C `for`-Schleife!

Vergleich zu C

C:

```c
int sum = 0;
for (int i=0; i<n; i++)
  if (data[i]%2==0) sum += data[i]*data[i];
```

Funktional lesbar vs. imperativ – gleiche Performance nach Optimierung.

Rust für Embedded und High-Performance

`no_std` – Rust ohne Standardbibliothek

```rust
#![no_std]
#![no_main]
use core::panic::PanicInfo;

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}

#[cortex_m_rt::entry]
fn main() -> ! {
    let p = stm32f4::take().unwrap();
    let gpioa = &p.GPIOA;
    gpioa.odr.modify(|_, w|
        w.odr5().set_bit()); // LED an
    loop { cortex_m::asm::wfi(); }
}
```

Die drei Bibliotheksebenen:

`core` – immer verfügbar, kein Allokator
`alloc` – benötigt Allokator (`Vec`, `Box`)
`std` – benötigt OS (Dateien, Threads)

Embedded-Ökosystem:

- `cortex-m-rt` – ARM Cortex-M Runtime
- `embedded-hal` – HAL (Trait-basiert)
- `svd2rust` – typsichere Register-APIs
- `defmt` / `probe-rs` – Logging/Debug

Vorteil gegenüber C

Register-Zugriffe sind *typsicher*:
kein `(volatile uint32_t*)0x4001...`
sondern `gpioa.odr.modify(|_,w| ...)`

unsafe und FFI

Was **unsafe** erlaubt:

1. Rohe Pointer dereferenzieren
2. `unsafe`-Funktionen aufrufen
3. Mutable statics verändern
4. `unsafe` Traits implementieren
5. Auf `union`-Felder zugreifen

```rust
// Hardware-Register direkt
const GPIOA_ODR: *mut u32
    = 0x4002_0014 as *mut u32;
unsafe {
    let val = core::ptr::read_volatile(
                    GPIOA_ODR);
    core::ptr::write_volatile(
        GPIOA_ODR, val | (1 << 5));
}
```

FFI – Aufruf von C-Code:

```rust
extern "C" {
    fn abs(input: i32) -> i32;
}
let x = unsafe { abs(-42) }; // 42

// Rust-Funktion für C exportieren
#[no_mangle]
pub extern "C" fn rust_add(
    a: i32, b: i32) -> i32
{
    a + b  // Kein unsafe nötig!
}
```

Unsafe ≠ unsicher

`unsafe` markiert die wenigen Stellen, an denen der Programmierer dem Compiler Invarianten garantiert.
Best Practice: Unsafe in Safe wrappen!

Threads mit Ownership:

```rust
use std::thread;
use std::sync::{Arc, Mutex};

let data = Arc::new(Mutex::new(vec![0u32;
    10]));

let handles: Vec<_> = (0..4).map(|id| {
    let data = Arc::clone(&data);
    thread::spawn(move || {
        let mut d = data.lock().unwrap();
        d[id] = id as u32 * 10;
    })
}).collect();

for h in handles { h.join().unwrap(); }
```

`Send` und `Sync`

`Send` – Typ darf zwischen Threads transferiert werden

`Sync` – Typ darf von mehreren Threads gleichzeitig referenziert werden

⇒ Automatisch abgeleitet, bei Verletzung: Compilerfehler

Data Races – unmöglich!

Borrowing-Regeln + `Send`/`Sync` verhindern Data Races zur *Compilezeit*. In C: Data Races sind undefined Behavior, werden vom Compiler *nicht* erkannt.

Für Embedded (bare-metal):

- Interrupt-sichere `Mutex` via `critical-section`
- RTIC Framework für Echtzeit

Performance: Zero-Cost Abstractions in der Praxis

Beispiel: Summierung gerader Quadrate

```rust
// Rust: funktionale Kette
fn sum_even_sq(data: &[i32]) -> i64 {
    data.iter()
        .filter(|&&x| x % 2 == 0)
        .map(|&x| (x as i64) * (x as i64))
        .sum()
}
```

```c
// C: äquivalente Schleife
int64_t sum_even_sq(const int32_t *d,
                    size_t n) {
    int64_t sum = 0;
    for (size_t i = 0; i < n; i++)
        if (d[i] % 2 == 0)
            sum += (int64_t)d[i] * d[i];
    return sum;
}
```

Beide erzeugen mit `-O2`/`-release` *identischen* Maschinencode (SIMD-vektorisiert).

Warum Zero-Cost?

- Monomorphisierung: Generics werden zu konkreten Funktionen aufgelöst
- Inlining: Closures werden inline expandiert
- Loop Fusion: Iterator-Ketten werden zu einer Schleife zusammengefasst
- LLVM-Backend: gleiche Optimierungen wie Clang/GCC

Benchmarks

Rust und C liefern in CPU-intensiven Benchmarks nahezu identische Ergebnisse (LLVM).

Vorteile Rust:

- Bounds Checks eliminierbar durch Iteratoren
- Aliasing-Infos für Optimizer (`&mut`)
- Kein UB → aggressivere Optimierung

Zusammenfassung

Zusammenfassung: Von C nach Rust

Neue Sprachkonzepte:

- Ownership & Borrowing: Speichersicherheit ohne GC, zur Compilezeit geprüft
- Pattern Matching: erschöpfende Fallunterscheidung mit `match`
- Algebraische Datentypen: Enums mit Daten, `Option`, `Result`
- Traits: typsichere Interfaces mit statischem und dynamischem Dispatch
- Generics: Monomorphisierung statt `void*`-Casts
- Closures & Iteratoren: funktionale Programmierung als Zero-Cost Abstraction

Sicherheitsmechanismen:

- Kein Null-Pointer (`Option<T>`)
- Kein Undefined Behavior im Safe Rust
- Data-Race-Freiheit durch `Send`/`Sync`
- Explizites `unsafe` für Hardware-Zugriff
- Fehlerbehandlung über Typsystem (`Result`)

Embedded & High-Performance:

- `no_std` für Bare-Metal-Targets
- `embedded-hal` für portierbare Treiber
- C-äquivalente Performance (LLVM)
- FFI für bestehenden C-Code
- Wachsendes Ökosystem (RTIC, Embassy, probe-rs)

Fragen?